

Anonymous Self-Stabilizing Distributed Algorithms for Connected Dominating Set in a Network Graph*

Wayne Goddard and Pradip K Srimani
School of Computing
Clemson University
Clemson, SC 29634-0974
{goddard, srimani}@cs.clemson.edu

Abstract

A self-stabilizing algorithm is a distributed algorithm where there is neither coordination nor initialization but the network achieves some global state. A connected dominating set (CDS) of a graph is a subset S of the nodes such that the subgraph induced by S is connected and every other node in the graph is adjacent to some node of S . A CDS is suitable as a spine or subset for communication. We provide and analyze two versions of a self-stabilizing algorithm for creating a good CDS. The better version is based on the construction of a breadth-first spanning tree with large internal degree and then discarding the leaves.

1 Introduction

The concept of self-stabilization, first proposed by Dijkstra [1], has proven an effective and cost-effective paradigm for localized state-based computation to implement distributed algorithms, especially in networks with resource-constrained nodes like sensor or ad hoc networks. The objective of self-stabilization (as opposed to fault masking) is to recover from failure in a reasonable time and without intervention by any external agency. Self-stabilization is based on two basic ideas: first, the code executed by a node is incorruptible (as if written in a fault-resilient memory) and transient faults affect only data values; second, the goal system behavior can be checked by evaluating some predicate of the system state variables.

In a self-stabilizing algorithm, every node has a set of local variables whose contents specify the

local state of the node. The state of the entire system, called the *global state*, is the union of the local states of all the nodes in the system. Each node has only a partial view of the global state and this view depends on the connectivity of the system. Furthermore, there is no synchronization, not even a common starting point. Yet, the system arrives at a desirable global final state (legitimate state).

The intrinsic algorithmic power of this paradigm—to design fault-tolerant protocols for maintaining global predicates using only local knowledge at nodes—has already been demonstrated. For example, a minimal spanning tree must be maintained to minimize latency and bandwidth requirements of multicast/broadcast messages or to implement echo-based distributed algorithms [2]; or a minimal dominating set must be maintained to optimize the number and the locations of the resource centers in a network. The authors of [3] proposed a self-stabilizing algorithm to compute a breadth-first spanning tree in a graph. This algorithm assumes the existence of a distinguished node, serving as a pre-determined root node of the spanning tree.

Another useful object in communication is a *connected dominating set*. A connected dominating set can serve as the communication backbone in the network, because its domination property ensures that every node is either in the set or adjacent to (some node in) the set, and its connectivity property guarantees any two nodes can message each other via a series of adjacent nodes in the set. Approximation algorithms to find a connected dominating set were given in [4, 5, 6].

A self-stabilizing algorithm is specified as a collection of rules for each node. Each rule has a trigger *condition* and an *action*. The trigger condition is a boolean predicate based on the states of the node and its neighbors, and the action specifies a change

*This work has been supported by NSF grant # CCF-0832582

in the state of its variables. A node is said to be *privileged* if the trigger condition on one of its rules is satisfied. Further, in order to consider the worst-case scenario, a self-stabilizing algorithm is assumed to face an adversarial daemon. We consider here a *distributed* daemon: this daemon picks an arbitrary subset of the privileged nodes to *move* (i.e., execute the rule's action) at each step.

It is difficult to construct a self-stabilizing algorithm for minimal connected dominating set, due to its nature of global predicates. Our objective in this paper is to propose two versions of a self-stabilizing algorithm for connected dominating sets in a given network. The algorithm is based on the construction of a breadth-first spanning tree and then discarding the leaves. The second version tries to boost the degree of internal nodes, thereby increasing the number of leaves, and decreasing the number of nodes that remain. We show the correctness of the two algorithms and provide brief experimental (simulation) results to show the effectiveness of the second heuristic.

We note that self-stabilizing algorithms for connected dominating set were recently given by both Jain and Gupta [7] and by Kamei and Kakugawa [8, 9]. Unlike their work, we allow both anonymous nodes and a distributed daemon. Neither their algorithms nor our proposal achieve a connected dominating set that is guaranteed to be minimal.

2 System Model & Performance Metrics

A network is modeled by an undirected graph $G = (V, E)$, where V is a set of n nodes and E is a set of edges. If $i \in V$, then $N(i)$, its *neighborhood*, denotes the set of nodes to which i is adjacent; every node $j \in N(i)$ is called a *neighbor* of node i . The distance between any two nodes in the graph G is the number of edges along a minimum path between the two nodes in G . Throughout this paper we assume G is connected and $n > 1$.

A set S , $S \subseteq V$, is called a *dominating* set if $N(i) \cap S \neq \emptyset$ for every $i \in V - S$. A dominating set S is called minimal when there does not exist a node $v \in S$ such that the set $S' = S - \{v\}$ is a dominating set. A dominating set is called a *connected dominating set (CDS)* if the subgraph induced by S is connected; minimality is defined similarly. Computing a CDS of minimum cardinality is NP-hard. And while self-stabilizing algorithms exist for computing a minimal dominating set (see for

example [10]), no self-stabilizing algorithm is known to compute a minimal CDS; the predicate seems to be locally non-computable.

2.1 Performance Measures

The obvious measure of a CDS of a given graph is its size (number of nodes). When the CDS is used to implement the backbone of the network, one might consider other measures. For example, if one uses the CDS to relay messages, then the *average distance* between two nodes of the CDS plays an important role. If this average is smaller, a message from one node to another will on the average go through fewer intermediate nodes and hence the probability of failure will decrease; on the other hand, it is possible to have two distinct CDS both of same size but different average distance; it may also be possible to get a CDS of smaller average distance with a relatively small increase in size.

3 Version 1: PruneTree

Given an undirected graph $G = (V, E)$, a spanning tree of G is a subset of $n - 1$ edges that connects all the nodes. We have a unique node r in the graph to serve as the root node of the tree; any node can determine if it is the root node or not. The *level* of a node in the spanning tree is the distance of the node from the unique root node as measured in the spanning tree. A spanning tree is a *breadth-first spanning (BFS)* tree iff the level of each node in the tree is equal to the distance between the node and the root node in G . In the algorithm, we assume that each node knows some value N that definitely exceeds the number of nodes in G .

The basic idea is to take a spanning tree and throw away the leaves. What remains must be connected and every leaf is adjacent to a remaining node, and so what remains is a CDS. We will improve the algorithm in a later section. However, since it helps the understanding to do the full analysis on this simpler version, and since we can show polynomial-time convergence for this version, we do the analysis and description now.

Each node i maintains the following data:

- A pointer variable P_i that stores the parent of node i in the spanning tree. When $i = r$, i.e., the node i is the root node r , we set $P(r) = r$.
- A nonnegative integer variable L_i that stores the level of node i in the spanning tree. Note

that L_i also denotes the distance of the node i from the root node r .

- A positive integer counter variable c_i , which is incremented modulo $\mathcal{N}$.
- A Boolean flag b_i that indicates node i is a member of the CDS iff $b_i = 1$.

In a legitimate global state, the state on each node is as follows:

Definition 1 A global system state is legitimate when

- (1) for the root node r , $P_r = r, L_r = 0$;
- (2) for any node $i \neq r$, $P_i = j \in N(i), L_i = L_j + 1$; and
- (3) the subset of nodes $\{i \mid b_i = 1\}$ is a CDS of the graph G .

It is immediate that in a legitimate global state the pointer variables P_i at each node i together form a BFS tree of G . Note that the definition of legitimate does not depend on the values of c_i .

For simplicity, we define a function M_i that returns the subset of neighbors of a node i with minimum L -value. In a legitimate state, the level of a node in a BFS tree is a nonnegative integer less than $\mathcal{N}$.

Definition 2 $m_i \stackrel{\text{def}}{=} \min\{L_k \mid k \in N(i)\}$. Further, $M_i \stackrel{\text{def}}{=} \{j \in N(i) \mid L_j = m_i\}$. For a node i in any state (legitimate or illegitimate), M_i denotes the subset of its neighbors with the minimum level. Note that M_i is not a stored variable at node i ; the node computes M_i from its local knowledge (its state and its neighbors' states).

Using this we can define the rules for constructing a BFS tree. See Figure 1.

BR: (Root node)
 if $(P_r \neq r) \vee L_r \neq 0$
 then $P_r := r; L_r := 0$

BG: (General node ($i \neq r$))
 if $(P_i = j \notin M_i) \vee (L_i \neq \min(L_j + 1, \mathcal{N} - 1))$
 then $\begin{cases} P_i := k \in M_i \text{ (arbitrarily);} \\ L_i := \min(L_k + 1, \mathcal{N} - 1) \end{cases}$

Figure 1: CDS Version 1: Rules BR and BG

Further we define:

R0: if $\delta_i > 0 \wedge b_i = 0$
 then $b_i := 1$
 else if $\delta_i = 0 \wedge b_i = 1$
 then $b_i := 0$

R1: if $(\nexists j \in N(i) \text{ s.t. } c_j = c_i + 1 \bmod \mathcal{N})$
 then $\begin{cases} \text{if } (i = r) \wedge (i \text{ privileged for BR}), \\ \text{then execute BR,} \\ \text{else if } (i \neq r) \wedge (i \text{ privileged for BG}), \\ \text{then execute BG;} \\ c_i := c_i + 1 \bmod \mathcal{N} \end{cases}$

Figure 2: Version 1 of the CDS Algorithm

Definition 3 $\delta_i \stackrel{\text{def}}{=} |\{j \in N(i) \mid P_j = i\}|$; That is, δ_i denotes the number of children of the node i in the spanning tree; again δ_i is computed at node i with its local knowledge.

The pseudocode of the full algorithm is given in Figure 2. The rules BR and BG construct a BFS tree. For a node i , if there is no neighboring node j with $c_j = c_i + 1$, then it is privileged to move. If it is privileged and chosen by the daemon, node i will do two things. First, node i checks if it is also privileged by the associated BFS tree rule (BR or BG); if so, the node executes that rule. Second, the node increments its c_i value.

When the system enters into a legitimate global state, the protocol does not actually terminate; the system makes transitions between legitimate states since only the counter variables c_i go on changing without violating the legitimacy of the system states. (The counter variables do not affect the BFS tree.) We say that a node is *B-stable* if the condition of the applicable rule BR or BG is false.

3.1 Analysis

Lemma 1 There will always be some node privileged for rule R1.

Proof: Suppose no node is privileged for rule R1. Then every node has a neighbor whose counter is one more than its counter, modulo $\mathcal{N}$. Consider any counter c_i . Then it must have a neighbor with value $c_i + 1$, which has a neighbor with value $c_i + 2$, and so on. But since the number of nodes is less than the number of possible counter values, we have a contradiction. $\square$

Theorem 1 *In any sequence of at least $\mathcal{N}^3$ moves by the daemon, every node executes rule R1 at least once.*

Proof: Assume R1 is not executed at node i . Thus c_i stays unchanged. Then we claim that:

If node j is at distance d from node i , then rule R1 can be executed on j at most $d \times (\mathcal{N} - 1)$ times.

The proof is by induction on d . Assume first that $d = 1$; that is j is a neighbor of i . Each time R1 is executed on j , c_j will be increased by 1. Since the calculations of c_j are in arithmetic modulo $\mathcal{N}$, it will reach $c_i - 1$ in at most $\mathcal{N} - 1$ increments. After that, R1 cannot be executed on j unless c_i changes first.

In general, assume the claim is true for $d = t$; i.e., for any two nodes i and j that are distance t away, if no R1 is executed on i , rule R1 can be executed on j at most $t \times (\mathcal{N} - 1)$ times. Assume now $d = t + 1$; for any node k that is distance $t + 1$ away from i , there is a node $j \in N(k)$ such that j is distance t away from i .

Let $F_s(j, k) = c_j - c_k \bmod n$, as of time T_s . Each time R1 is executed on j , $F_s(j, k)$ is increased by 1; Each time R1 is executed on k , $F_s(j, k)$ is decreased by 1. So for any two time instances T_1 and T_2 , let A_j and A_k denote the maximum number of executions on j and k between T_1 and T_2 , if no executions of R1 are made on i . We have: $|A_k - A_j| \leq |F_2(j, k) - F_1(j, k)|$.

If j does not move, c_k can increase to at most $c_j - 1$, hence $F_s(j, k) \geq 1$; if k does not move, c_j can increase to at most $c_k - 1$, hence $F_s(j, k) \leq \mathcal{N} - 1$. In addition, it is only in the initial state that $c_j = c_k$ is possible. Once j or k has moved, the value of $F(j, k)$ will be in $[1, \mathcal{N} - 1]$ and will remain in this range. Thus, $A_k \leq A_j + |F_2(j, k) - F_1(j, k)| \leq A_j + (\mathcal{N} - 1)$. By assumption, $A_j \leq t \times (\mathcal{N} - 1)$, so $A_k \leq A_j + (\mathcal{N} - 1) = (t + 1) \times (\mathcal{N} - 1)$. That is, for any k that is distance $t + 1$ away from i , if no R1 is executed on i , rule R1 can be executed on k at most $(t + 1) \times (\mathcal{N} - 1)$ times. This proves the claim.

Now, every node is at most distance $\mathcal{N} - 1$ away from i . So, if R1 is not executed on i , for any other node j , rule R1 can be executed on it at most $(\mathcal{N} - 1)^2$ times. For all $n - 1$ nodes except i , the number of executions is bounded by $(\mathcal{N} - 1)^3$. $\square$

Theorem 2 *If a node is at distance d from the root, then from time $(d + 1)\mathcal{N}^3$ onwards it is B-stable.*

Proof: We prove the theorem by induction on d . Simultaneously, we also prove that from that time onwards, any node that is distance more than d from the root has an L -value more than d .

Consider first $d = 0$. By time $\mathcal{N}^3$, the root has executed R1 at least once. So if it was not B-stable initially, it has executed BR and is now B-stable. Further, any non-root node with L -value of 0 has executed BG and now has L -value at least 1.

In general, consider time $(d + 1)\mathcal{N}^3$. By the inductive hypothesis, we know that by time $d\mathcal{N}^3$, every node within distance $d - 1$ of the root is B-stable. Furthermore, every other node has L -value at least d . Now, between time $d\mathcal{N}^3$ and $(d + 1)\mathcal{N}^3$, every node executes R1. So any node i at distance d that is not B-stable executes BG, and thereafter i has L -value d and points to a node at distance $d - 1$. Furthermore, any node j at distance more than d has minimum L -value in its neighborhood at least d . So after executing BG if privileged for it, node j has L -value now at least $d + 1$, as required. $\square$

Thus within time $\mathcal{N}^4$ all nodes will be B-stable and will remain so. That is, the algorithm stabilizes within time $\mathcal{N}^4$.

4 Version 2: PruneGoodTree

Our second version introduces an important refinement with the objective of building the tree more intelligently. As before, we assume we have a designated node r to be used as the root of the tree. We now create a “bushy” BFS tree T from the root. In the bushy BFS tree, nodes point to a neighbor with smaller distance to the root (level in the spanning tree), as before. However, ties are broken by the node pointing to the parent with the most current children. (If still tied, then ties are broken randomly.) Again one obtains a CDS by discarding the leaves of the tree T .

In this version, the number of children is added as a stored variable:

- A nonnegative integer variable D_i that stores the number of children of the node (that is, δ_i).

We define a new predicate SM_i [Superior Minimum] at each node i :

Definition 4 $SM_i \stackrel{\text{def}}{=} \{j \in M_i \mid D_j = \min_{k \in M_i} \{D_k\}\}$; that is, SM_i denotes the subset of

minimum-level neighbors of node i with the largest number of claimed children in the spanning tree.

Then the BG-rule is modified accordingly. (See Figure 3.)

BG: (General node ($i \neq r$))
 if $(P_i = j \notin SM_i) \vee (L_i \neq \min(L_j + 1, \mathcal{N} - 1))$
 then $\begin{cases} P_i := k \in SM_i \text{ (arbitrarily);} \\ L_i := \min(L_k + 1, \mathcal{N} - 1) \end{cases}$

Figure 3: CDS Version 2: Modified BG rule

Further, rule R0 is modified to keep the value of D_i current. (See Figure 4.)

R0: if $D_i \neq \delta_i$, then $D_i := \delta_i$
 (and keep b_i current as before)

Figure 4: CDS Version 2: Augmented R0 rule

Complete pseudocode of Version 2 can be obtained from Version 1 by replacing its rules BG and R0 by the modified rules.

The analysis of this version is similar in parts to the analysis of the first version. The proofs of Lemma 1 and Theorem 1 remain unchanged. The proof of Theorem 2 carries over in that one can show that a node becomes non-privileged under the original BG rule:

Theorem 3 *If a node i is at distance $d > 0$ from the root, then from time $(d+1)\mathcal{N}^3$ onwards, it always points to some node in M_i that is at distance $d - 1$ from the root.*

However, it is possible for a node to alter its parent several times. Nevertheless, one can readily show that the algorithm will stabilize eventually:

Theorem 4 *From some point on, every node is B-stable.*

Proof: Suppose there is always some node that is not B-stable. Since there are only a finite number of values for the local variables, that means that some global state must occur infinitely often, and the BG/BR rule must be executed infinitely often.

Let D denote the largest value of the children counter D_i that is ever set (as a result of rule R0) between two consecutive occurrences of the same global state. Say it occurs at node i . At that point, node i has D children pointing to it. For such a

child to change from pointing to i , it must see another node, say j , with a higher value of D_j , at a later time. But that contradicts our choice of D . $\square$

It is unclear whether the adversarial daemon can force the stabilizing period to take exponential time.

5 Simulation

We performed a primitive simulation to see if the second algorithm performs significantly better than the first. In future work we will compare this with the existing algorithms.

5.1 Comparative Experiments

To compare the two algorithms, we simplified matters. We used a random serial daemon, ignoring the counters c_i . That is, at each move we randomly picked one node privileged for BG or R0.

Further, we used a unit disk graph for our network. This was obtained from a *random geometric graph* model that entails placing n nodes at random in a unit circle in the plane, each with a transmission range of λ . Two nodes are adjacent iff they are within (Euclidean) distance λ of each other. Ellis et al. [11] showed that there is a sharp threshold for the connectivity of such a random geometric graph. Based on that, we set $\lambda = f\sqrt{\ln n}/n$ where f is a factor that we can vary and n is the number of nodes. We then randomly generated graphs until we determine that we have a connected graph; by their result, if $f > 1$ then almost surely the graph is connected, so we did not encounter many disconnected ones.

5.2 Results

In this paper, we provide data for the case $f = 1$ of the random geometric graph above. See Figure 5. As the factor f increases, the data is qualitatively the same. In all cases Version 2 outperforms Version 1, as expected, except that as f increases, the networks become denser and thus the size and average distance of the CDS all decrease. In future simulation work we hope to compare this algorithm directly with the other published self-stabilizing algorithms.

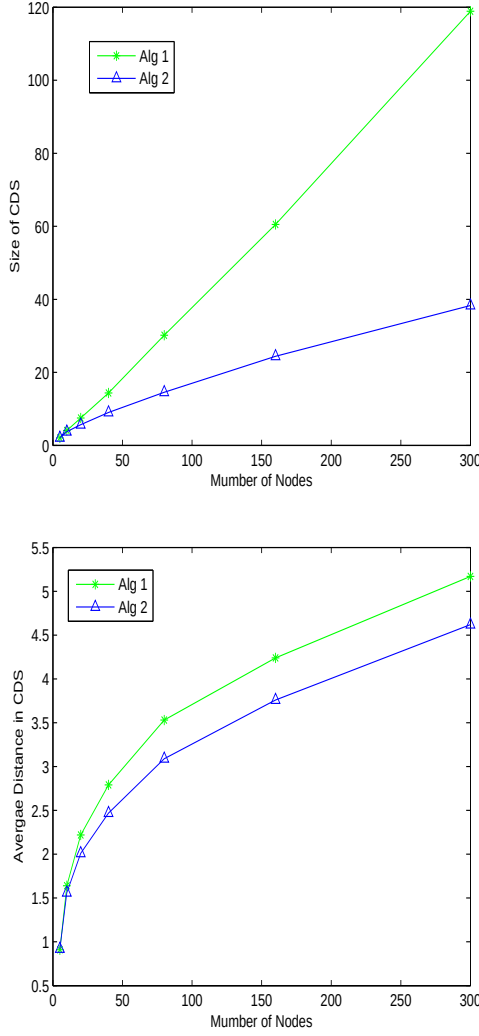


Figure 5: Simulation Comparison of the Two Algorithms

6 Conclusion

We provided the first self-stabilizing algorithm for connected dominating set that handles both anonymous nodes and a distributed daemon simultaneously. The first version of the algorithm stabilizes in a polynomial number of moves. The simulation confirms that the heuristic of constructing a spanning tree taking into account the number of children at each node improves the connected dominating set that is obtained, and suggests that this heuristic might be useful in other tree algorithms.

References

- [1] E.W. Dijkstra. Self stabilizing systems in spite of distributed control. *Comm. ACM*, 17:643–644, 1974.
- [2] H. Attiya and J. Welch. *Distributed Computing: Fundamentals, Simulations and Advanced Topics*. McGraw Hill, 1998.
- [3] S. Dolev, A. Israeli, and S. Moran. Self-stabilization of dynamic systems assuming only read/write atomicity. *Distrib. Comput.*, 7:3–16, 1993.
- [4] S. Guha and S. Khuller. Approximation algorithms for connected dominating sets. *Algorithmica*, 20:374–387, 1998.
- [5] P.-J. Wan, K.M. Alzoubi, and O. Frieder. Distributed construction of connected dominating set in wireless ad hoc networks. *MONET*, 9(2):141–149, 2004.
- [6] Fei Dai and Jie Wu. An extended localized algorithm for connected dominating set formation in ad hoc wireless networks. *IEEE Trans. Parallel Distrib. Systems*, 15(10):908–920, 2004.
- [7] A. Jain and A. Gupta. A distributed self-stabilizing algorithm for finding a connected dominating set in a graph. In *PDCAT*, pages 615–619. IEEE Computer Society, 2005.
- [8] S. Kamei and H. Kakugawa. A self-stabilizing distributed approximation algorithm for the minimum connected dominating set. In *IPDPS*, pages 1–8. IEEE, 2007.
- [9] S. Kamei and H. Kakugawa. A self-stabilizing approximation for the minimum connected dominating set with safe convergence. In *OPODIS*, volume 5401 of *LNCS*, pages 496–511. Springer, 2008.
- [10] Z. Xu, S.T. Hedetniemi, W. Goddard, and P.K. Srimani. A synchronous self-stabilizing minimal domination protocol in an arbitrary network graph. In *Distributed Computing—IWDC 2003, Kolkata, India, Springer LNCS:2918*, pages 26–32, 2003.
- [11] R.B. Ellis, J.L. Martin, and C. Yan. Random geometric graph diameter in the unit ball. *Algorithmica*, 47(4):421–438, 2007.